Pregătire AA

1) Demonstrați că $o(g(n)) \cap \omega(g(n)) = \emptyset$

Varianta 1

$o(g(n)) = \{ f(n) / (\forall) c \in \mathbb{R}^*, \exists n_0 \in \mathbb{N}^* \text{ a.î. } 0 \le f(n) \le c \cdot g(n), (\forall) n \ge n_0 \}$

$\omega(g(n)) = \{ f(n) / (\forall) c \in \mathbb{R}^*, \exists n_0 \in \mathbb{N}^* \text{ a.î. } 0 \le c \cdot g(n) < f(n), (\forall) n \ge n_0 \}$

Presupunem prin reducere la absurd

$o(g(n)) \cap \omega(g(n)) \neq \emptyset \Rightarrow \exists h(n)$ a.î.

$h(n) \in o(g(n))$ și $h(n) \in \omega(g(n))$.

Conform definițiilor:

$h(n) \in o(g(n)) \Rightarrow (\forall) c \in \mathbb{R}_+^*, \exists n_0 \in \mathbb{N}^*$

$0 \le h(n) < c \cdot g(n), (\forall) n \ge n_0$

$h(n) \in \omega(g(n)) \Rightarrow (\forall) c \in \mathbb{R}_+^*, \exists n_0 \in \mathbb{N}^*$

$0 \le c \cdot g(n) < h(n), (\forall) n \ge n_0$

Contradicție, se ajunge la $h(n) < h(n)$.

Varianta 2

$f(n) = O(g(n)) \Rightarrow \lim_{n\to+\infty} \frac{f(n)}{g(n)} \neq \infty$

$f(n) = \Omega(g(n)) \Rightarrow \lim_{n\to+\infty} \frac{f(n)}{g(n)} \neq 0$

$f(n) = \Theta(g(n)) \Rightarrow \lim_{n\to+\infty} \frac{f(n)}{g(n)} \nearrow 0 \;\text{ sau }\; \neq \infty$

$\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$

$f(n) = o(g(n)) \Rightarrow \lim_{n\to+\infty} \frac{f(n)}{g(n)} = 0$

$f(n) = \omega(g(n)) \Rightarrow \lim_{n\to+\infty} \frac{f(n)}{g(n)} = +\infty$

$o(g(n)) = O(g(n)) \setminus \Theta(g(n))$

$\omega(g(n)) = \Omega(g(n)) \setminus \Theta(g(n))$

Metoda substituției

$$T(n) = \begin{cases} k_1, & n=1 \\ 2T(n/2) + k_2, & n>1 \end{cases}$$ obținut din Master

Alegem $T(n) = \Theta(n) \Rightarrow \exists c_1, c_2 \in \mathbb{R}_+^*$

$\exists n_0 \in \mathbb{N}^*$ astfel încât

$$c_1 \cdot n \leq T(n) \leq c_2 \cdot n, \ (\forall)\ n \geq n_0$$

Caz de bază

$$n=1 \Rightarrow \underbrace{c_1 \leq k_1 \leq c_2}_{T(1)} \ (*), \ n_0 = 1$$

Pas de inducție $n/2 \to n$

Ipoteză de inducție: $\exists c_1, c_2 \in \mathbb{R}_+^*, \exists n_0 \in \mathbb{N}^*$ a.î.

$$c_1 \cdot \frac{n}{2} \leq T\left(\frac{n}{2}\right) \leq c_2 \cdot \frac{n}{2}, \ (\forall)\ n \geq n_0 \ / \cdot 2$$

$$c_1 \cdot n \leq 2T\left(\frac{n}{2}\right) \leq c_2 \cdot n, \ (\forall)\ n \geq n_0$$

$$c_1 \cdot n + k_2 \leq \underbrace{2T\left(\frac{n}{2}\right) + k_2}_{T(n)} \leq c_2 \cdot n + k_2, \ (\forall) n \geq n_0$$

$$\leq c_2 n$$

$$c_1 \cdot n + k_2 \leq T(n) \leq c_2 \cdot n + k_2, \ (\forall)\ n \geq n_0$$

$c_1 n \leq$ ⟶ $\Rightarrow k_2 = 0$ contradicție

Nu rezolvăm încă cu Master. Rezolvare cu metoda iterației sau a arborelui de recurență.

Putem lua: $T(n) = \Theta(2n-1)$

↓ obținut din iterație

$$T(n) = 3T(n/7) + k_1 \cdot n \quad | \cdot 3^0 \rightarrow$$ Metoda iterației

$$T(n/7) = 3T(n/7^2) + k_1 \frac{n}{7} \quad | \cdot 3^1$$

— — — — — — — — ;

$$T(n/7^k) = 3 \cdot T(n/7^{k+1}) + k_1 \cdot \frac{n}{7^k} \quad | \cdot 3^k$$

$$T(n) = 3^{k+1} \cdot T(n/7^{k+1}) + k_1 \cdot n \cdot \sum_{i=0}^{k} \left(\frac{3}{7}\right)^i$$

$$\frac{n}{7^{k+1}} = 1 \Rightarrow n = 7^{k+1} \Rightarrow k+1 = \log_7 n$$

$$\Rightarrow T(n) = 3^{\log_7 n} \cdot T(1) + k_1 \cdot n \cdot \frac{\left(\frac{3}{7}\right)^{k+1} - 1}{\frac{3}{7} - 1} =$$

$$= 3^{\log_7 n} + k_1 \cdot n \cdot \left[\left(\frac{3}{7}\right)^{\log_7 n} - 1\right] \cdot \frac{7}{-4} =$$

$$= 3^{\log_7 n} + k_1 \cdot n \cdot \left(1 - \frac{3^{\log_7 n}}{7^{\log_7 n}}\right) \cdot \frac{7}{4} =$$

$$= 3^{\log_7 n} + k_1 \cdot n \left(1 - \frac{3^{\log_7 n}}{n}\right) \cdot \frac{7}{4} =$$

$$= 3^{\log_7 n} + \frac{7}{4} \cdot k_1 \cdot n - \frac{7}{4} k_1 \cdot n / \frac{3^{\log_7 n}}{n} =$$

$$= 3^{\log_7 n} \underbrace{\left(1 - \frac{7}{4} k_1\right)}_{\text{const}} + \underbrace{\frac{7}{4} k_1 \cdot n}_{\text{const}} \Rightarrow$$

$$T(n) = \Theta(3^{\log_7 n} + n) = \Theta(\underbrace{n}_{\log_7 3 < 1}^{\log_7 3} + n)$$

$$= \Theta(n)$$

Care este complexitatea spațială pt. algoritmul:

```
Alg(n) {
    rezultat = 0
    for i = 1 ... n
        for j = 1 ... n
            rezultat = rezultat + n
}
```

$rezultat \leq n^3$ $i \leq n$ $j \leq n$

Valorile maxime ale variabilelor (fără datele de intrare)

$O(\log n^3)$
$O(3 \log n)$

$O(\log n)$

$O(\log n)$

Complexitatea $O(3\log n + \log n + \log n) = O(\log n)$

```
HAlg(x, m, n) {
    i = choice({1, ..., m}) --- O(1)
    j = choice({1, ..., n}) --- O(1)
    if i*j > x
        print(i, j)
        success
    fail
}
```

O(1)

Tipul TList

TList := nil / cons (T, TList)

a) Constructori de bază

nil : → TList (constructor extern) → constructor nular

cons : T × TList → TList → constructor intern

b) Operatori

head : TList \ {nil} → T

tail : TList \ {nil} → TList

null : TList → {0, 1}

c) Axiome

head (cons (x, L)) = x

tail (cons (x, L)) = L

null (nil) = 1

Algoritmul concat

```
concat (l1, l2)
    if (l1 = nil)
        return l2
    else
    if(l1 = cons(x, l3))
        return cons(x, concat(l3, l2))
```

Dem. prin inducție structurală

$(\forall)\ l_1 \in tList,\ concat(l_1, nil) = l_1$

Cazul de bază: aplicat pt. toți constructorii externi ai TDA

$l_1 = nil$

$concat(l_1, nil) = concat(nil, nil) =$

$nil = l_1$

Pas de inducție: aplicat pt. toți constr. interni ai TDA-ului $L \to cons(x, L)$

Ipoteză de inducție: $concat(L, nil) = L$

$concat(cons(x, L), nil) = cons(x, L)$?

↳ trebuie demonstrat

$concat(concat(x, L), nil) =$

$= cons(x, \underbrace{concat(L, nil)}) = cons(x, L)$

Ip. de inducție

Problema TSP pt. un graf complet $G = (V, E)$ și vrem să verificăm dacă $\exists$ un ciclu de cost cel mult k.

$V = \{1, 2, 3, 4\}$ $\quad 1, 2, 3, 4, 1$

$1, 3, 2, 4, 1$

Generez permutări de V

Generare permutări

```
NAlg(V, E, k) {
    n = |V|
    perm = [0, 0, ..., 0] // n elemente
    for i = 1 ... n                          --- O(n)
        x = choice(V)                        --- O(1)
        for j = 1, ..., i-1                  --- O(n)
            if perm[j] == x                  --- O(1)
                fail
        perm[i] = x                          --- O(1)
```

* perm conține o permutare a lui V

```
    cost = 0
    for i = 1 ... n-1                        } O(n)
        cost = cost + E[perm[i]][perm[i+1]]  }
    cost = cost + E[perm[n]][perm[1]]
```

if cost c=k
↳ success
fail

} O(1)

} ⇒ algoritmul are complexitate polinomială

(*) Cum pot verifica dacă este k costul celui mai mic ciclu?

Formăm o altă funcție și verificăm dacă există cicluri de cost 1, 2, ---, k-1. Dacă există, se returnează fail, altfel dacă există și ciclu de cost k, se returnează success.